B-Splines & Atomic Physics

Complete Self-Study Guide — Assignments 4, 5 & 6

Python · NumPy · SciPy · johntfoster/bspline · Poisson · Hydrogen · DFT

---

*A single integrated reference covering everything from Python basics to a working self-consistent-field DFT code. Includes a full anatomy of the **johntfoster/bspline** library — every class, method, and utility function explained and mapped to the exact line of code where you call it.*

*Colour key: **Blue headers** = physics / maths content **Purple headers** = johntfoster/bspline library*

Contents

PART A — FOUNDATIONS

1. Weekend Roadmap & Time Plan

You have roughly **two full days** plus a polishing day. The table below is a realistic schedule assuming you follow this guide linearly.

Time block	Goal	Sections here
Sat morning (3 h)	Python + NumPy + B-spline theory Write knot vector, plot all splines	2, 3, 6, 7–9
Sat afternoon (4 h)	Assignment 4 — Poisson collocation Test uniform sphere against analytical	4, 10
Sat evening (2 h)	Assignment 4 — shell + hydrogen cases LU reuse, plots	10
Sun morning (4 h)	Assignment 5 — hydrogen eigenvalue Build H and S via Gauss quadrature	4, 11
Sun afternoon (3 h)	Assignment 5 — multiple ■ values, wave functions Assignment 6 — SCF skeleton	11, 12
Sun evening (3 h)	Assignment 6 — helium convergence Neon orbital energies	12
Monday (4 h)	Polish code · make plots · build presentation	13, 14, 15

■ *Assignments 4 and 5 share 90 % of the same B-spline code. Build the knot vector and Bspline object once in a shared file; import it in both assignments.*

2. Python Crash Course — the Pieces You Actually Need

2.1 Variables and Types

Python variables hold references to objects. You will use four types almost exclusively: **float**, **int**, **numpy array**, and **dict** (for storing named results).

```
x    = 3.14          # float
n    = 10            # integer
flag = True          # boolean
name = "hydrogen"    # string (rarely needed in physics code)

# Dictionary — useful for storing orbital results keyed by (n, ell)
energies = {}
energies[(1, 0)] = -0.5      # store ground-state energy
print(energies[(1, 0)])      # -0.5
```

2.2 Lists vs NumPy Arrays — Use Arrays for All Numerics

```
py_list = [1, 2, 3]              # Python list — no fast maths
import numpy as np
arr = np.array([1.0, 2.0, 3.0])   # NumPy array — vectorised maths

arr * 2          # array([2., 4., 6.])   — no loop needed
arr ** 2         # array([1., 4., 9.])
np.sqrt(arr)     # array([1., 1.41, 1.73])
np.sum(arr)      # 6.0
```

2.3 Loops and range()

```
for i in range(5):       # i = 0, 1, 2, 3, 4
    print(i)

for i in range(1, 10, 2):   # 1, 3, 5, 7, 9
    print(i)

# Enumerate gives index + value simultaneously
r_vals = np.linspace(0, 10, 5)
for idx, r in enumerate(r_vals):
    print(idx, r)           # 0 0.0 / 1 2.5 / ...
```

2.4 Zero-Based Indexing

```
a = np.array([10., 20., 30., 40., 50.])
a[0]       # 10.0  — first element
a[-1]      # 50.0  — last element
a[1:4]     # [20., 30., 40.]  — slice, end is exclusive
a[::2]     # [10., 30., 50.]  — every second element
```

3. NumPy Essentials

3.1 Creating Arrays

```
import numpy as np
np.zeros(10)               # [0. 0. 0. ...]  length 10
np.zeros((5, 5))           # 5×5 matrix of zeros
np.linspace(0, 10, 50)    # 50 evenly spaced points from 0 to 10
np.eye(5)                  # 5×5 identity matrix
np.unique(t)               # sorted unique values — used for knot points
```

3.2 Matrix Operations — Core of All Three Assignments

```
A = np.zeros((4, 4))
A[0, 0] = 1.0              # set element
A[2, :] = np.array([1,2,3,4])   # fill entire row
A[:, 3] = np.array([0,1,2,3])   # fill entire column

b = np.array([1., 2., 3., 4.])

x = np.linalg.solve(A, b)  # solve A x = b
y = A @ b                   # matrix-vector multiply  (same as np.dot(A, b))

# Matrix-matrix multiply: used with collmat results
A_plot = B.collmat(r_values)   # shape (N_r, n_basis)
phi    = A_plot @ c            # shape (N_r,)  — reconstructed function
```

3.3 Boolean Indexing — for Charge Densities

```
r   = np.linspace(0, 15, 200)
R   = 5.0
rho = np.zeros_like(r)
rho[r <= R] = 3.0 / (4 * np.pi * R**3)   # uniform inside sphere
# rho[r > R] is already 0.0
```

3.4 Integration with trapz

```
# Quick check: does 4*pi * int rho r^2 dr = Q ?
check = 4 * np.pi * np.trapz(rho * r**2, r)
print(f"Charge = {check:.4f}")   # should equal Q = 1.0
```

4. SciPy — Linear Solvers and Eigenvalue Problems

4.1 LU Factorisation — Solve Once, Reuse Many Times (Assignment 4)

Poisson's equation gives the same matrix A for all three charge distributions. LU-factorising once and calling lu_solve three times saves 2/3 of the work.

```
from scipy.linalg import lu_factor, lu_solve

lu, piv = lu_factor(A)          # factorise ONCE — O(N^3)
c_sphere = lu_solve((lu, piv), rhs_sphere)   # O(N^2) each
c_shell  = lu_solve((lu, piv), rhs_shell)
c_hydro  = lu_solve((lu, piv), rhs_hydro)
```

4.2 Generalised Eigenvalue Problem (Assignment 5 & 6)

The Galerkin formulation gives **H c = E S c** where both H and S are symmetric. Use **eigh** (not eig) — it exploits symmetry and returns real eigenvalues in sorted order.

```
from scipy.linalg import eigh

eigenvalues, eigenvectors = eigh(H_mat, S_mat)
# eigenvalues[i]     — i-th energy (Hartree), sorted ascending
# eigenvectors[:,i]  — corresponding coefficient vector c_i

# Bound states have negative energy:
bound = eigenvalues[eigenvalues < 0]
print(f"Found {len(bound)} bound states")
print(f"Ground state: {bound[0]:.6f} Ha  (exact for H: -0.5 Ha)")
```

4.3 Gauss-Legendre Quadrature — Exact Integration of B-Spline Products

B-splines are piecewise polynomials of degree p. Products of two basis functions are polynomials of degree 2p. Gauss-Legendre with **p+1** points per interval integrates these exactly.

```
from numpy.polynomial.legendre import leggauss

ngl = 5                              # points per interval (p+1 = 4 is minimum)
gl_z, gl_w = leggauss(ngl)           # nodes and weights on [-1, 1]

# Transform to interval [a, b]:
def gl_on(a, b):
    x = 0.5*(b-a)*gl_z + 0.5*(b+a)  # points in [a, b]
    w = 0.5*(b-a)*gl_w               # scaled weights
    return x, w

# Test: integrate r^2 from 0 to 5
x, w = gl_on(0, 5)
print(np.sum(w * x**2))   # should be 5^3/3 = 41.667
```

5. matplotlib — Plotting Results

```
import matplotlib.pyplot as plt
import numpy as np

r = np.linspace(0.01, 10, 300)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(r, -1.0/r,  "b-",  linewidth=1.5, label="V = -1/r  (Coulomb)")
ax.plot(r, np.exp(-2*r), "r--", label="rho ~ exp(-2r)  (H 1s)")
ax.set_xlabel("r  [Bohr]")
ax.set_ylabel("value")
ax.set_xlim(0, 10);  ax.set_ylim(-3, 1.1)
ax.legend();  ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("result.png", dpi=150)   # save for report
plt.show()

# Multiple subplots — useful for comparing assignments
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
axes[0].set_title("Sphere")
axes[1].set_title("Shell")
axes[2].set_title("Hydrogen 1s")
```

PART B — B-SPLINES

6. B-Spline Theory from First Principles

6.1 What is a Knot Sequence?

A knot sequence is a non-decreasing list t[0] ≤ t[1] ≤ ... ≤ t[N-1]. Think of it as the grid skeleton on which your basis functions live. Repeated values at the boundaries force the splines to be exactly zero there.

```
# Manual construction (assignment PDF convention, k=4):
k     = 4                          # order in assignment notation
r_max = 20.0
inner = np.linspace(0, r_max, 12)  # 12 distinct physical points
t = np.concatenate([
    np.repeat(inner[0],   k),      # k=4 copies of 0
    inner[1:-1],                   # 10 interior points
    np.repeat(inner[-1],  k)       # k=4 copies of r_max
])
# len(t) = 4 + 10 + 4 = 18
# n_splines = len(t) - k = 18 - 4 = 14
```

6.2 The Recursive Definition (Cox–de Boor)

Order 1: $B_{i,1}(x) = 1$ if $t_i \le x < t_{i+1}$, else 0

Order k: $B_{i,k}(x) = [(x-t_i)/(t_{i+k-1}-t_i)]\, B_{i,k-1}(x) + [(t_{i+k}-x)/(t_{i+k}-t_{i+1})]\, B_{i+1,k-1}(x)$

■ *Convention: 0/0 = 0. This handles repeated knots safely.*

6.3 Key Properties

- **Local support:** $B_{i,k}(x)$ is non-zero only for $x \in [t_i, t_{i+k}]$. This gives banded matrices — only k–1 splines overlap at any point.
- **Partition of unity:** $\sum B_{i,k}(x) = 1$ everywhere between t_{first} and t_{last}.
- **Smoothness:** $B_{i,k}$ is C^{k-2}. For k=4: C^2 — twice differentiable everywhere except at knots with multiplicity > 1.
- **Boundary BCs:** With k repeated boundary knots, only B_0 is nonzero at the left end and only B_{last} at the right end. Dropping these two splines enforces zero boundary conditions automatically.

6.4 Derivatives

First derivative:

$$dB_{i,k}/dx = (k-1)\,[\, B_{i,k-1}/(t_{i+k-1}-t_i) - B_{i+1,k-1}/(t_{i+k}-t_{i+1})\,]$$

Second derivative: Apply the first-derivative formula twice. The library handles this automatically via *diff(order=2)*.

7. The johntfoster/bspline Library — Repository Layout

The repository at **https://github.com/johntfoster/bspline** provides a NumPy implementation of the Cox–de Boor algorithm with two Python files that are all you need:

```
johntfoster/bspline/
■
■■■ bspline/               ← the Python package (what pip installs)
■   ■■■ __init__.py        ← makes 'import bspline' work
■   ■■■ bspline.py         ← class Bspline  +  memoize decorator
■   ■■■ splinelab.py       ← augknt, aptknt, aveknt, knt2mlt, spcol
■
■■■ test/                  ← unit tests (good to read for expected behaviour)
■■■ README.md              ← usage examples
■■■ setup.py               ← pip packaging
```

Install

```
pip install bspline          # from PyPI (simplest)

# OR from your forked repo:
git clone https://github.com/YOUR-USERNAME/bspline.git
cd bspline
pip install -e .             # editable: source changes take effect immediately
```

Standard imports at the top of every assignment file

```
import numpy as np
import bspline
import bspline.splinelab as splinelab
from scipy.linalg import lu_factor, lu_solve, eigh
from numpy.polynomial.legendre import leggauss
import matplotlib.pyplot as plt
```

8. The Order Convention: Library p vs Assignment k (READ THIS FIRST)

The single most common source of bugs when using this library with the assignment PDF. They use different names for the same thing.

Property	Assignment PDF (de Boor notation)	johntfoster/bspline (CS notation)
Main symbol	k	p (called 'order' in __init__)
Polynomial degree	k – 1	p
Cubic splines	k = 4	p = 3
Boundary repeats per end	k copies	(p+1) copies total (augknt adds p, knots has 1)
Number of splines (N total knots)	N – k	len(t) – p – 1
Min order for 2nd derivative	k ≥ 4	p ≥ 3

■ KEY: Assignment says k=4 → pass p=3 to the library. Count splines as len(t) – p – 1, not N – k.

```
# THE TRANSLATION:
k_assign = 4                    # assignment notation
p_lib    = k_assign - 1         # = 3  → pass this to Bspline() and augknt()

# Verify they give the same number of splines:
inner = np.linspace(0, 20, 30)

# Assignment manual approach:
t_manual = np.concatenate([np.repeat(inner[0], k_assign),
                           inner[1:-1],
                           np.repeat(inner[-1], k_assign)])
n_sp_assign = len(t_manual) - k_assign

# Library approach:
t_lib    = splinelab.augknt(inner, p_lib)
n_sp_lib = len(t_lib) - p_lib - 1

print(n_sp_assign, n_sp_lib)   # both should print 32  ← same!
```

9. The johntfoster/bspline API — Every Method Explained

9.1 splinelab.augknt(knots, order) — Build the knot vector [FIRST CALL]

Source (verbatim): return np.array([knots[0]]*order + list(knots) + [knots[-1]]*order)

What it does: Prepends *order* (= p) copies of the first knot and appends *order* copies of the last knot. Combined with the existing endpoint, each boundary gets p+1 = k copies total.

```
import numpy as np
import bspline.splinelab as splinelab

p     = 3                                    # cubic (k=4 in assignment)
inner = np.array([0., 5., 10., 15., 20.])    # 5 distinct physical points

t = splinelab.augknt(inner, p)
print(t)
# [ 0.  0.  0.  0.  5.  10.  15.  20.  20.  20.  20.]
#  ^-- p=3 prepended --^               ^-- p=3 appended --^
# Total length = 5 + 2*3 = 11
# n_basis = len(t) - p - 1 = 11 - 3 - 1 = 7
```

■ *LIBRARY: This is the ONLY function from splinelab you MUST call. Everything else is optional.*

9.2 bspline.Bspline(knot_vector, order) — Create the basis object

What it does: Stores the knot vector and degree p. Immediately runs two dummy evaluations at x=0 to warm the memoisation cache.

Create it ONCE and reuse — the cache lives on the instance.

```
import bspline
import bspline.splinelab as splinelab

p     = 3
inner = np.linspace(0, 30, 40)
t     = splinelab.augknt(inner, p)

B = bspline.Bspline(t, p)           # ← CREATE ONCE at the top of your script
n_basis = len(t) - p - 1
print(f"Basis has {n_basis} functions")

# Reuse B for all matrix building, reconstruction, and plotting.
# DO NOT create a new Bspline() inside a loop — cache starts empty each time.
```

9.3 B(x) — All basis function values at one point x

Returns: 1-D array of length n_basis. Element i = $B_i(x)$. At most p+1 = 4 elements are non-zero (local support).

When to use it: Evaluating a spline-expanded function at a single point, or building matrices one point at a time.

```
vals = B(2.5)                  # shape (n_basis,)
# vals[i] = B_i(2.5)
# Most are 0.0 — only ~4 are nonzero near x=2.5

# Reconstruct phi(r) at one point from coefficients c:
c = np.random.randn(n_basis)
```

```
phi_at_2_5 = np.sum(B(2.5) * c)

# Memoised: calling B(2.5) a second time is free (cached).
```

9.4 B.d(x) — First derivatives of all basis functions [FAST PATH]

Returns: 1-D array. Element i = dB_i/dx at x.

This is the fast path. Uses a modified recursion (replaces the numerator in the last step with ±p). Memoised, same speed as B(x).

```
dvals = B.d(3.7)              # shape (n_basis,)
# dvals[i] = dB_i/dx  at x = 3.7

# ■■ Assignment 5: kinetic energy contribution at Gauss point x, weight w ■■
kinetic_contrib = 0.5 * w * dvals[j] * dvals[i]

# Use B.d(x), NOT B.diff(order=1)(x) — they give the same result
# but d() is faster because it takes the internal fast path.
```

9.5 B.diff(order=n) — Any-order derivative as a callable

Returns: A lambda function f(x) that evaluates the n-th derivative of ALL basis functions at x. Each differentiation step doubles the number of internal terms (2^n terms total). Practical up to n=3.

```
D2 = B.diff(order=2)          # second-derivative function
d2vals = D2(4.1)              # shape (n_basis,)
# d2vals[i] = d^2 B_i / dx^2  at x = 4.1

# ■■ Assignment 4: fill the collocation matrix row by row ■■■■■■■■■■
D2 = B.diff(order=2)
for row, ri in enumerate(collocation_points):
    A[row, :] = D2(ri)        # entire row from one call
    rhs[row]  = -ri * 4*np.pi * rho(ri)

# For order > p, diff() returns identically zero (correct behaviour):
D5 = B.diff(order=5)
print(D5(0.3))   # array of zeros
```

■ *diff(order=1)(x) and d(x) give identical results, but d(x) is faster. Always use d(x) for first derivatives.*

9.6 B.collmat(tau, deriv_order=0) — The collocation matrix [KEY]

Returns: 2-D array of shape (len(tau), n_basis). Element [i, j] = $D^{deriv\_order}\ B_j(tau_i)$.

This is the most important method for your assignments. It builds an entire matrix block in one vectorised call — far faster than looping over collocation points yourself.

```
# ■■ Most common patterns ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

# 1. Interior collocation matrix (second derivatives) — Assignment 4
all_knots = np.unique(t)
tau_int   = all_knots[1:-1]           # interior physical knots

A2 = B.collmat(tau_int, deriv_order=2)
# Shape: (N_int-2, n_basis)
# A2[i, j] = d^2 B_j/dr^2  at r = tau_int[i]
# → Paste into rows 1 to N_int-2 of your system matrix
```

```
# 2. Boundary condition rows
row_bc0  = B.collmat([all_knots[0]],  deriv_order=0)  # shape (1, n_basis)
row_bcR  = B.collmat([all_knots[-1]], deriv_order=0)  # shape (1, n_basis)
# row_bc0  will be [1, 0, 0, ...] because only B_0 nonzero at r=0
# row_bcR  will be [..., 0, 1]   because only B_last nonzero at r_max

# 3. Reconstruct function on a dense grid — MATRIX MULTIPLY
r_plot = np.linspace(0.01, r_max, 400)
A_plot = B.collmat(r_plot, deriv_order=0)    # shape (400, n_basis)
phi    = A_plot @ c                           # shape (400,)  — phi(r) values

# 4. Galerkin matrices — Assignment 5
#    Evaluate ALL basis functions at ALL Gauss points in one call
x_gauss, w_gauss = gl_on(a, b)             # ngl Gauss points on [a, b]
Bvals  = B.collmat(x_gauss, deriv_order=0) # shape (ngl, n_basis)
dBvals = B.collmat(x_gauss, deriv_order=1) # shape (ngl, n_basis)
# Bvals[:, j]  = B_j at all Gauss points
# dBvals[:, j] = dB_j/dr at all Gauss points
```

■ KEY: Use A_plot @ c (matrix-vector multiply) instead of a loop over basis functions when reconstructing the potential on a dense grid. It is 100x faster.

9.7 splinelab utilities: aptknt · aveknt · knt2mlt · spcol

These are rarely needed for your assignments but documented for completeness.

```
# aptknt(tau, order) — knot vector fitted to collocation sites
# Use when you have specific sites where the equation must be satisfied
# and want the knot vector to match them. For your work, augknt is simpler.
t_apt = splinelab.aptknt(np.linspace(0,20,15), p=3)

# aveknt(t, k) — running average of k successive knot entries
# Used internally by aptknt. Rarely called directly.
avg = splinelab.aveknt(np.array([0.,1.,2.,3.,4.]), k=3)
# → [1.0, 2.0, 3.0]  (averages of [0,1,2], [1,2,3], [2,3,4])

# knt2mlt(t) — count how many times each knot has appeared before
m = splinelab.knt2mlt(np.array([0.,0.,0.,0.,5.,10.,20.,20.,20.,20.]))
# → [0, 1, 2, 3, 0, 0, 0, 1, 2, 3]

# spcol(knots, order, tau) — collocation matrix with multiplicity-aware derivatives
# Like B.collmat() but automatically uses higher derivatives at repeated sites.
# Only needed if you want to enforce both value AND derivative at the same point.
A = splinelab.spcol(t, p=3, tau=np.unique(t)[1:-1])  # same as B.collmat(tau)
```

9.8 The memoize Cache — Why You Create Bspline Only Once

The **memoize** decorator (from bspline.py) caches the return value of any call based on the arguments. The first call computes and stores the result. All subsequent calls with the same argument retrieve it instantly.

```
# From bspline.py (simplified):
# @memoize is applied to both __call__ and d:
#
# First call:  B(2.5)  → runs full Cox-de Boor recursion → stores in cache
# Second call: B(2.5)  → returns cached value, no recomputation
#
# The cache key is the argument value (xi must be hashable, i.e. a float).
# The cache is stored on the Bspline INSTANCE.
```

```
# CORRECT: create once, reuse
B = bspline.Bspline(t, p)
for xi in gauss_points:
    vals = B(xi)       # second iteration: cache hit for any repeated xi

# WRONG: creates new object each iteration — cache is always empty
for xi in gauss_points:
    B_tmp = bspline.Bspline(t, p)   # DON'T DO THIS
    vals = B_tmp(xi)
```

■ *The memoize cache stores floats as keys. Passing numpy scalars vs Python floats can sometimes create separate cache entries. Convert Gauss points to Python floats if you see unexpected slowness: float(xi).*

PART C — ASSIGNMENTS

10. Assignment 4 — Poisson Collocation

10.1 The Physics

We want the electrostatic potential V(r) from a spherically symmetric charge ρ(r). With the substitution $\phi(r) = r \times V(r)$, the 3-D Poisson equation reduces to a 1-D second-order ODE (with $4\pi\varepsilon_0 = 1$):

$$\mathbf{d^2\phi/dr^2 = -4\pi\, r\, \rho(r)}$$

Boundary conditions: $\phi(0) = 0$ (regularity at origin); $\phi(r \to \infty) = Q$ (total charge).

10.2 Method — Step by Step

- Build knot vector t with augknt.
- Create Bspline object B.
- Get collocation matrix of second derivatives: A2 = B.collmat(tau, deriv_order=2).
- Assemble system matrix: row 0 = BC at r=0; interior rows = A2; last row = BC at r_max.
- Build RHS: row 0 = 0; interior = –r■×4πρ(r■); last = Q.
- LU-factorise once; solve for each charge distribution.
- Reconstruct: phi = B.collmat(r_plot) @ c; V = phi / r.

10.3 Library Calls Mapped to Each Step

Step	Library call	Returns
Build knot vector	splinelab.augknt(inner, p=3)	1-D array t
Create basis	bspline.Bspline(t, p=3)	Bspline object B
Second-deriv matrix	B.collmat(tau_int, deriv_order=2)	shape (N-2, n_basis)
BC row at r=0	B.collmat([0.], deriv_order=0)	shape (1, n_basis) = [1,0,...,0]
BC row at r_max	B.collmat([r_max], deriv_order=0)	shape (1, n_basis) = [...,0,1]
Solve system	lu_factor(A) then lu_solve((lu,piv), rhs)	coefficient vector c
Reconstruct phi	B.collmat(r_plot) @ c	phi values on r_plot

10.4 Complete Working Code

```
import numpy as np
import bspline
import bspline.splinelab as splinelab
from scipy.linalg import lu_factor, lu_solve
import matplotlib.pyplot as plt

# ■■ Parameters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
p_lib = 3          # library degree  (= assignment k-1 = 4-1)
r_max = 20.0
Q     = 1.0        # total charge
N_pts = 35         # distinct physical knot points

# ■■ Step 1: Knot vector ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
inner = np.linspace(0, r_max, N_pts)
t     = splinelab.augknt(inner, p_lib)
n_basis = len(t) - p_lib - 1        # number of B-splines

# ■■ Step 2: Create basis object (ONCE) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
B = bspline.Bspline(t, p_lib)

# ■■ Step 3: Collocation points (distinct interior knots) ■■■■■■■■■■
all_knots = np.unique(t)             # length N_pts
tau_int   = all_knots[1:-1]         # interior, length N_pts-2

# ■■ Step 4: Assemble system matrix ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
A_sys = np.zeros((n_basis, n_basis))

# Row 0: phi(r=0) = 0
A_sys[0, :] = B.collmat([all_knots[0]],  deriv_order=0)

# Interior rows: d^2 phi / dr^2 = -r * 4*pi*rho
A_int = B.collmat(tau_int, deriv_order=2)     # shape (N_pts-2, n_basis)
A_sys[1:len(tau_int)+1, :] = A_int

# Last row: phi(r_max) = Q
A_sys[-1, :] = B.collmat([all_knots[-1]], deriv_order=0)

# ■■ Step 5: LU factorise ONCE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
lu, piv = lu_factor(A_sys)

# ■■ Step 6: Solve for each distribution ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
R = 5.0
def rho_sphere(r):
    V = (4/3)*np.pi*R**3
    return np.where(r <= R, Q/V, 0.0)

def rho_shell(r, R1=4., R2=8.):
    V = (4/3)*np.pi*(R2**3 - R1**3)
    return np.where((r >= R1) & (r <= R2), Q/V, 0.0)

def rho_hydrogen(r):
    return 4.0 * np.exp(-2*r)       # 4*pi*|psi_1s|^2, psi_1s = exp(-r)/sqrt(pi)

def build_rhs(rho_func):
    rhs = np.zeros(n_basis)
    rhs[0]  = 0.0                   # phi(0) = 0
    for idx, ri in enumerate(tau_int):
        rhs[idx+1] = -ri * 4*np.pi * rho_func(ri)
    rhs[-1] = Q                     # phi(r_max) = Q
    return rhs

c_sphere = lu_solve((lu, piv), build_rhs(rho_sphere))
c_shell  = lu_solve((lu, piv), build_rhs(rho_shell))
c_hydro  = lu_solve((lu, piv), build_rhs(rho_hydrogen))

# ■■ Step 7: Reconstruct and plot ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
r_plot = np.linspace(0.02, r_max, 500)
A_plot = B.collmat(r_plot, deriv_order=0)    # shape (500, n_basis)

V_sphere = (A_plot @ c_sphere) / r_plot
V_shell  = (A_plot @ c_shell)  / r_plot
V_hydro  = (A_plot @ c_hydro)  / r_plot

# ■■ Analytical check for sphere ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
V_exact = np.where(r_plot <= R,
                   Q/R * (1.5 - 0.5*(r_plot/R)**2),
                   Q/r_plot)
```

```
print(f"Max error: {np.max(np.abs(V_sphere - V_exact)):.2e}")

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, V, title in zip(axes,
                        [V_sphere, V_shell, V_hydro],
                        ["Sphere", "Shell", "Hydrogen 1s"]):
    ax.plot(r_plot, V, "b-", label="Numerical")
    ax.set_title(title); ax.set_xlabel("r [Bohr]"); ax.grid(alpha=0.3)
axes[0].plot(r_plot, V_exact, "r--", label="Analytical"); axes[0].legend()
plt.tight_layout(); plt.savefig("poisson_all.png", dpi=150)
```

10.5 Analytical Solutions

Sphere (R, Q=1): $V = Q/(2R)(3–r^2/R^2)$ for $r \leq R$; $V = Q/r$ for $r > R$

Shell (R_1, R_2, Q=1): $V = \text{const} = Q/(4\pi R_1)$ for $r < R_1$; $V = Q/r$ for $r > R_2$

Hydrogen 1s: $V(r) = 1/r – e^{-2r}(1/r + 1)$

11. Assignment 5 — Hydrogen Eigenvalue Problem

11.1 Physics and Galerkin Formulation

The radial Schrödinger equation (Hartree units, ■ = m_e = 1):

$$[-½ d²/dr² + ■(■+1)/(2r²) - Z/r] P_{n■} = E P_{n■}$$

Expand $P = \sum c_i B_i$, multiply by B_j, integrate → generalised eigenvalue problem **H c = E S c**:

$$H_{ij} = ½\int B'_j B'_i dr + ■(■+1)/2 \int B_j r^{-2} B_i dr - Z\int B_j r^{-1} B_i dr$$

$$S_{ij} = \int B_j B_i dr$$

11.2 Library Calls for Matrix Construction

Matrix element	Library calls	Note
Kinetic ½∫B'j B'i	B.collmat(x_gl, deriv_order=1) → dBvals[:,j] * dBvals[:,i]	Use d() for speed at single points; collmat is better for many points
Centrifugal ■(■+1)/2 ∫Bj r■² Bi	B.collmat(x_gl, deriv_order=0) → Bvals[:,j] / x_gl² * Bvals[:,i]	Avoid evaluating at r=0!
Coulomb −Z ∫Bj r■¹ Bi	B.collmat(x_gl, deriv_order=0) → Bvals[:,j] / x_gl * Bvals[:,i]	
Overlap ∫Bj Bi	B.collmat(x_gl, deriv_order=0) → Bvals[:,j] * Bvals[:,i]	Gauss weights sum to interval length

11.3 Complete Working Code

```
import numpy as np
import bspline
import bspline.splinelab as splinelab
from scipy.linalg import eigh
from numpy.polynomial.legendre import leggauss
import matplotlib.pyplot as plt

# ■■ Parameters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
p_lib = 3;  Z = 1;  ell = 0
r_max = 30.0;  N_pts = 50;  ngl = 5

# ■■ Knot vector and basis ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
inner = np.linspace(0, r_max, N_pts)
t     = splinelab.augknt(inner, p_lib)
B     = bspline.Bspline(t, p_lib)      # CREATE ONCE
n_basis = len(t) - p_lib - 1
n_act   = n_basis - 2                  # drop first and last (BCs)

# ■■ Gauss-Legendre quadrature ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
gl_z, gl_w = leggauss(ngl)
t_uniq = np.unique(t)

H_mat = np.zeros((n_act, n_act))
S_mat = np.zeros((n_act, n_act))

for m in range(len(t_uniq) - 1):
    a, b = t_uniq[m], t_uniq[m+1]
    if a == b: continue
```

```
    x_pts = 0.5*(b-a)*gl_z + 0.5*(b+a)    # Gauss pts on [a, b]
    w_pts = 0.5*(b-a)*gl_w

    # ■■ Evaluate ALL basis functions at ALL Gauss points (vectorised)
    Bv  = B.collmat(x_pts, deriv_order=0)  # shape (ngl, n_basis)
    dBv = B.collmat(x_pts, deriv_order=1)  # shape (ngl, n_basis)

    for ii in range(n_act):
        i = ii + 1    # actual basis index (active = 1 to n_basis-2)
        for jj in range(n_act):
            j = jj + 1
            Bj = Bv[:,j]; Bi = Bv[:,i]
            dBj=dBv[:,j]; dBi=dBv[:,i]

            kin  = np.sum(w_pts * 0.5 * dBj * dBi)
            cent = np.sum(w_pts * 0.5*ell*(ell+1) / x_pts**2 * Bj * Bi)
            coul = np.sum(w_pts * (-Z) / x_pts * Bj * Bi)
            over = np.sum(w_pts * Bj * Bi)

            H_mat[ii,jj] += kin + cent + coul
            S_mat[ii,jj] += over

# ■■ Solve generalised eigenvalue problem ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
evals, evecs = eigh(H_mat, S_mat)

print(f"Bound states for ell={ell}, Z={Z}:")
for idx, E in enumerate(evals[:8]):
    if E >= 0: break
    n = idx + ell + 1
    E_ex = -Z**2 / (2*n**2)
    print(f"  n={n}: E={E:.6f} Ha  exact={E_ex:.6f} Ha  err={abs(E-E_ex):.1e}")

# ■■ Reconstruct and normalise ground state ■■■■■■■■■■■■■■■■■■■■■■■■■
r_plot = np.linspace(0.01, r_max, 500)
A_pl   = B.collmat(r_plot, deriv_order=0)   # shape (500, n_basis)
c_gs   = evecs[:, 0]                         # ground state coefficients
P_gs   = A_pl[:, 1:-1] @ c_gs               # shape (500,)  — skip BC splines
norm   = np.trapz(P_gs**2, r_plot)
P_gs  /= np.sqrt(norm)

P_exact = 2*r_plot*np.exp(-r_plot)           # H 1s: P(r) = 2r exp(-r)
plt.figure(figsize=(7,4))
plt.plot(r_plot, P_gs,    "b-",  label="B-spline")
plt.plot(r_plot, P_exact, "r--", label="Exact 1s")
plt.xlabel("r [Bohr]"); plt.ylabel("P(r)"); plt.legend()
plt.title(f"Hydrogen radial function, ell={ell}")
plt.tight_layout(); plt.savefig("hydrogen_gs.png", dpi=150)
```

11.4 Exponential Knot Sequence (Better for Hydrogen)

Hydrogen wave functions decay as $e^{-r/n}$. An exponential knot sequence concentrates points near r=0 where the function changes most rapidly.

```
# Exponential inner knots — dense near origin, sparse far out
N_exp    = 40
r_min_in = 0.01
inner_exp = np.exp(np.linspace(np.log(r_min_in), np.log(r_max), N_exp))
inner_exp = np.concatenate([[0.0], inner_exp])   # prepend r=0
t_exp = splinelab.augknt(inner_exp, p_lib)
# This gives much better accuracy for n=3,4,5 states with fewer knots.
```

12. Assignment 6 — Self-Consistent Field for Many-Electron Atoms

12.1 The SCF Loop

Assignment 6 plugs Assignments 4 and 5 together inside an iteration loop. The library calls are identical — the only new code is computing ρ from wave functions and adding V_{ee} to the H matrix.

- Start: solve hydrogen-like eigenvalue (V_{ee}=0) for each occupied (n,■).
- Normalise orbitals $P_{n■}(r)$. Check: $4\pi\int\rho r^2 dr = N_{elec}$.
- Solve Poisson (Assign. 4 code, unchanged) for $V^{dir}_{ee}(r)$.
- Add Slater exchange: $V^{exch} = -3[3\rho/(8\pi)]^{1/3}$.
- Add V_{ee} term to H matrix inner loop (one extra np.sum per pair).
- Mix: $V_{ee} \leftarrow (1-\eta)V_{ee,new} + \eta V_{ee,old}$ with $\eta \approx 0.4$.
- Repeat until max $|\Delta E_{n■}| < 10^{-5}$ Ha (≈30 iterations for He, 60 for Ne).

12.2 Aufbau Orbital Filling

```
# Filling order for neutral atoms:
aufbau = [
    (1,0,2),  # 1s  — 2 electrons
    (2,0,2),  # 2s
    (2,1,6),  # 2p  — 6 electrons
    (3,0,2),  # 3s
    (3,1,6),  # 3p
    (4,0,2),  # 4s   ← fills BEFORE 3d (Madelung rule)
    (3,2,10), # 3d  — 10 electrons
    (4,1,6),  # 4p
]  # each tuple: (n, ell, max_electrons)

def get_config(Z_atom):
    remaining = Z_atom
    config = []
    for (n, l, cap) in aufbau:
        if remaining <= 0: break
        occ = min(remaining, cap)
        config.append((n, l, occ))
        remaining -= occ
    return config

print(get_config(2))    # He:  [(1,0,2)]
print(get_config(10))   # Ne:  [(1,0,2),(2,0,2),(2,1,6)]
print(get_config(19))   # K:   [... (4,0,1)]   — 19th electron in 4s, not 3d!
```

12.3 Charge Density, Exchange Potential, Total Energy

```
# ■■ Charge density from occupied orbitals ■■■■■■■■■■■■■■■■■■■■■■■■■■
def compute_rho(config, orbitals, r_grid):
    rho = np.zeros_like(r_grid)
    for (n, l, N_e) in config:
        P    = orbitals[(n,l)]                    # shape (len(r_grid),)
        norm = np.trapz(P**2, r_grid)
        P   /= np.sqrt(norm)                      # normalise
        rho += N_e * (P/r_grid)**2 / (4*np.pi)   # add contribution
    check = 4*np.pi * np.trapz(rho * r_grid**2, r_grid)
    assert abs(check - sum(N for _,_,N in config)) < 0.01, f"rho normalisation failed: {check:.3f}"
    return rho
```

```
# ■■ Slater exchange ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def V_exchange(rho):
    return -3.0 * (3.0 * np.maximum(rho, 1e-30) / (8.0*np.pi))**(1.0/3.0)

# ■■ Add V_ee to H matrix elements (inside interval loop) ■■■■■■■■■■■
from scipy.interpolate import interp1d
V_ee_fn = interp1d(r_grid, V_dir + V_exch, kind='linear',
                   fill_value=0., bounds_error=False)
# Inside the ii/jj loop:
V_ee_pts = V_ee_fn(x_pts)                    # V_ee at Gauss points
vee_contrib = np.sum(w_pts * Bv[:,j] * V_ee_pts * Bv[:,i])
H_mat[ii,jj] += vee_contrib

# ■■ Total energy (no double counting) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def total_energy(config, E_orbitals, orbitals, V_ee, r_grid):
    E_tot = 0.0
    for (n, l, N_e) in config:
        P  = orbitals[(n,l)]
        Vee_exp = np.trapz(P**2 * V_ee, r_grid)
        E_tot  += N_e * (E_orbitals[(n,l)] - 0.5 * Vee_exp)
    return E_tot
```

12.4 Convergence Check Values

Atom	Orbital	Start (no V_ee)	Converged (with V_ee)	Source
He (Z=2)	1s	–2.000 Ha	≈ –0.740 Ha	Assignment sheet
Ne (Z=10)	1s	–50.0 Ha	≈ –31.4 Ha	Assignment sheet
Ne (Z=10)	2s	–12.5 Ha	≈ –1.53 Ha	Assignment sheet
Ne (Z=10)	2p	–12.5 Ha	≈ –0.68 Ha	Assignment sheet

■ *He ionisation energy from this method: ≈ 0.70 Ha ≈ 19 eV. Experimental: 24.6 eV. The Slater exchange approximation is less accurate for atoms with few electrons.*

PART D — REFERENCE

13. Debugging Checklist

Assignment 4

- ■ V(r) matches 1/r outside the sphere analytically?
- ■ Row 0 of A enforces $\phi(0)=0$? (should be [1, 0, 0, ...])
- ■ Last row of A enforces $\phi(r\_max)=Q$?
- ■ RHS has both the factor r AND the factor 4π?
- ■ V = phi/r, not V = phi?
- ■ p_lib = 3 passed to augknt and Bspline (not k=4)?
- ■ collmat used with deriv_order=2 for interior rows?

Assignment 5

- ■ Active splines are indices 1 to n_basis-2 (drop first and last)?
- ■ Kinetic term uses dBj * dBi (integration by parts), NOT $d^2Bj * Bi$?
- ■ eigh called (not eig) — needs symmetric H and S?
- ■ Ground state energy within 10^{-4} of –0.5 Ha for hydrogen?
- ■ Coulomb term is –Z/r (negative — attractive)?
- ■ r_max large enough that $P(r\_max) \approx 0$?
- ■ Avoid evaluating $1/r^2$ or 1/r at r=0 (use Gauss points, never at 0)?

Assignment 6

- ■ Orbitals normalised before computing ρ? ($\int P^2 dr = 1$)
- ■ $4\pi\int \rho r^2 dr = N_{elec}$ (charge conservation)?
- ■ Poisson BCs: $\phi(0)=0$, $\phi(r\_max)=N_{occ}$? (not Q=1 as in Assign. 4)
- ■ Direct potential is positive at large r (repulsive from electrons)?
- ■ Exchange potential is negative everywhere (attractive correction)?
- ■ Mixing applied: $V_{new} = (1-\eta)V_{new} + \eta V_{old}$?
- ■ He converges to orbital energy $\approx$ –0.74 Ha after ~30 iterations?

14. What the Library Does NOT Do

The library handles the B-spline maths. Everything physics-specific is yours.

You write this yourself	Library provides this
Charge density ρ(r) definitions	augknt — knot vector construction
Boundary condition rows in matrix	Bspline(t,p) — basis object
RHS vector –r×4π×ρ	collmat() — matrix of values/derivatives
Call scipy to solve the system	B(x), d(x), diff(n) — pointwise evaluation
Gauss-Legendre quadrature loop	Memoisation for repeated evaluations
Galerkin H and S matrix assembly	plot(), dplot() — quick visualisation
Normalise eigenvectors from eigh	
Compute ρ from wave functions (A6)	
SCF convergence loop (A6)	
Slater exchange potential (A6)	

■ *The library does about 20% of the work — the elegant, fiddly B-spline maths. Your 80% is the physics: boundary conditions, PDEs, integrands, and iteration loops.*

15. Quick-Reference Formula Sheet

Hartree Atomic Units

Length: $a_0 = 0.529$ ■ | Energy: 1 Ha = 27.211 eV | ■ = $m_e = e = 4\pi\varepsilon_0 = 1$

Bound state energies of hydrogen: $E_n = -Z^2/(2n^2)$ Ha

Library Order Convention

Assignment k=4 ↔ Library p=3. n_basis = len(t) – p – 1. Active splines: indices 1 to n_basis–2.

Poisson (ϕ=rV, 4$\pi\varepsilon$■=1)

$d^2\phi/dr^2 = -4\pi r\rho(r)$. BC: $\phi(0)=0$, $\phi(\infty)=Q$. $V(r) = \phi(r)/r$.

Galerkin H Matrix Elements (Hartree units)

$H_{ij} = \frac{1}{2}\int B'_j B'_i dr + ■(■+1)/2\int B_j r^{-2} B_i dr - Z\int B_j r^{-1} B_i dr + \int B_j V_{ee} B_i dr$

Gauss-Legendre on [a,b]

$x = \frac{1}{2}(b-a)z + \frac{1}{2}(b+a)$; $w = \frac{1}{2}(b-a)w_{GL}$. Use p+1 = 4 points per interval minimum.

Slater Exchange

$V^{exch}(r) = -3[3\rho/(8\pi)]^{1/3}$ (negative, attractive)

SCF Mixing

$V_{new} \leftarrow (1-\eta)V_{new} + \eta V_{old}$, η=0.4. Converge when $\max|\Delta E_{n■}| < 10^{-5}$ Ha.

Total Energy (no double counting)

$E_{total} = \sum_i N_i [E_i - \frac{1}{2} ■P_i|V_{ee}|P_i■]$

You have everything you need. Start with augknt → Bspline → collmat for Assignment 4. Add d() and Gauss quadrature for Assignment 5. Wrap in a convergence loop for Assignment 6. Good luck!